

RE — Exercices de renforcement (Leçons 01 + 01b)

Exercices plus poussés qui combinent les concepts vus. Compile toujours avec `gcc -g -O0` pour pouvoir déboguer avec GDB.

```
cd notes/reverse-engineering/lecon
```

Exercice R1 — Tableau inversé (boucle + pointeurs)

Écris un programme qui :

- Déclare `int tab[] = {11, 22, 33, 44, 55}`
- Inverse le tableau **en place** (sans créer de second tableau)
- Affiche le résultat

Contrainte : utilise **uniquement des pointeurs** (pas d'index `tab[i]`).

Résultat attendu :

```
55 44 33 22 11
```

Indice RE : en assembleur, un swap en place utilise un registre temporaire (`mov tmp, [rax] / mov [rax], [rbx] / mov [rbx], tmp`).

► Solution

Exercice R2 — Compteur de voyelles (boucle + conditions + strings)

Écris une fonction `int compter_voyelles(char *s)` qui retourne le nombre de voyelles (a, e, i, o, u, majuscules incluses) dans une chaîne.

Teste avec :

```
"Reverse Engineering" → 6  
"AAAA" → 4  
"xyz" → 0
```

► Solution

Exercice R3 — Struct + pointeur de struct (struct + pointeurs)

Crée une struct `Cible` avec :

- `char ip[16]`
- `int port`
- `int ouvert` (0 ou 1)

Écris une fonction `void scanner(struct Cible *c)` qui :

- Met `ouvert` à 1 si le port est 21, 22, 80, ou 443
- Met `ouvert` à 0 sinon

Dans `main` :

- Crée un tableau de 4 cibles avec des ports différents
- Appelle `scanner` sur chacune via pointeur
- Affiche les résultats

Résultat attendu :

```
192.168.1.1:22    -> OUVERT
192.168.1.2:8080  -> FERME
192.168.1.3:80    -> OUVERT
192.168.1.4:3306  -> FERME
```

Indice RE : en asm, l'accès à un champ struct est un offset fixe depuis le pointeur de base (`mov eax, [rbx+16]` = accès au champ à l'offset 16).

► Solution

Exercice R4 — XOR brute-force (bitwise + boucle + strings)

Tu as intercepté un message chiffré avec un XOR à clé d'un seul byte (tu ne connais pas la clé).

```
unsigned char encrypted[] = {0x2d, 0x28, 0x3b, 0x3b, 0x38, 0x11, 0x64,
0x38, 0x69, 0x3b, 0x27};
int len = 11;
```

Écris un programme qui **brute-force** les 256 clés possibles (0x00 à 0xFF) et affiche chaque résultat. Trouve la clé qui produit un message lisible.

Indice : le message est en ASCII imprimable.

Résultat attendu (parmi les 256 lignes) :

```
cle 0x4c : allez_world  (non)
cle 0x5c : HELLO WORLD  (oui, ou similaire)
...
```

► Solution

Exercice R5 — Malloc + recopie manuelle (heap + pointeurs + strings)

Écris ta propre version de `strdup` :

```
char *mon_strdup(char *src);
```

- Calcule la longueur de `src` manuellement (pas de `strlen`)
- Alloue la mémoire avec `malloc`
- Copie byte par byte avec un pointeur (pas de `strcpy`)
- Retourne le nouveau pointeur

Teste avec :

```
char *copie = mon_strdup("metasploit");  
printf("%s\n", copie); // metasploit  
free(copie);
```

Indice RE : `strdup` est souvent inlinée par le compilateur. En asm, tu verras un `call malloc` suivi d'une boucle `mov byte [rdi], [rsi]`.

► Solution

Exercice R6 — Pointeurs de fonctions : dispatch table (fonctions + pointeurs + struct)

Simule un mini-interpréteur de commandes :

Crée un tableau de structs `Commande` :

```
struct Commande {  
    char nom[16];  
    void (*action)(void);  
};
```

Implémente 3 fonctions : `cmd_scan`, `cmd_exploit`, `cmd_exit` (chacune affiche un message).

Dans `main` :

- Demande une commande à l'utilisateur
- Parcours le tableau et appelle la bonne `action` via le pointeur de fonction
- Si commande inconnue → affiche "commande inconnue"

Résultat attendu :

```
> scan
[*] Scanning target...
> exploit
[*] Launching exploit...
> exit
[*] Bye
> blah
commande inconnue
```

Indice RE : les dispatch tables (vtables, jump tables) sont un pattern asm classique : `call [rax+offset]`.

► Solution

Exercice R7 — Décodeur multi-couches (XOR + cast + malloc + tout)

Un malware encode ses strings en deux couches :

1. XOR avec la clé `0x37`
2. Chaque byte est stocké inversé (le dernier byte d'abord)

Voici le payload encodé :

```
unsigned char payload[] = {0x45, 0x05, 0x51, 0x03, 0x56, 0x04, 0x42, 0x57,
0x44, 0x04};
int len = 10;
```

Écris un programme qui :

1. Inverse le tableau en place
2. XOR chaque byte avec `0x37`
3. Affiche le message décodé

Fais tout avec des pointeurs, malloc pour le buffer de travail, et free à la fin.

► Solution

Exercice R8 — Crackme final (combinaison totale)

Compile **sans regarder le code après**, puis cracke le binaire :

```
// crackme_final.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Auth {
    char user[32];
```

```
int level;
int (*check)(char *, char *);
};

int verify(char *input, char *secret) {
    return strcmp(input, secret) == 0;
}

int main() {
    struct Auth auth;
    strcpy(auth.user, "admin");
    auth.level = 3;
    auth.check = verify;

    char key[] = {0x70, 0x40, 0x57, 0x57, 0x74, 0x60, 0x52, 0x27};
    int len = 8;
    char *decoded = (char *)malloc(len + 1);
    int i;
    for (i = 0; i < len; i++)
        decoded[i] = key[i] ^ 0x13;
    decoded[len] = '\0';

    char input[50];
    printf("[%s@level%d] Password: ", auth.user, auth.level);
    scanf("%49s", input);

    if (auth.check(input, decoded))
        printf("Flag: RE_{struct_xor_fptr}\n");
    else
        printf("Access denied\n");

    free(decoded);
    return 0;
}
```

```
gcc -g -O0 -o crackme_final crackme_final.c
```

3 méthodes pour le résoudre :

1. **strings** — est-ce que le mot de passe est en clair ? Pourquoi pas ?
2. **GDB** — `break strcmp` → `x/s $rsi` pour lire le secret décodé
3. **Calcul** — XOR les bytes `key[]` avec `0x13` à la main ou en Python

Checkpoint

- ☐ R1 : tableau inversé avec pointeurs
- ☐ R2 : compteur de voyelles
- ☐ R3 : struct Cible + scanner
- ☐ R4 : XOR brute-force

- ☐ R5 : mon_strdup (malloc + copie manuelle)
- ☐ R6 : dispatch table (pointeurs de fonctions)
- ☐ R7 : décodeur multi-couches
- ☐ R8 : crackme final — trouvé par GDB ou calcul